SECRETARÍA DE EDUCACIÓN PÚBLICA

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE VERACRUZ.

MATERIA:

NOSQL

CARRERA:

SISTEMAS COMPUTACIONALES

PROYECTO:

UNIDAD 2 - SNACKVER

ALUMNOS:

JUAN VICENTE HERRERA VIVANCO

PROFESOR TITULAR:

JOSE NAHUM PRIETO VELAZQUEZ

FECHA DE ENTREGA:

14/10/2026

ÍNDICE

INDICE DE IMÁGENES ____ *3*

INDICE DE TABLAS ____ *4*

INTRODUCCIÓN ____ *5*

CONTENIDO: IMPLEMENTACION TECNICA ____ *6*

MIGRACIÓN DE SQL A MONGODB ATLAS ____ **6**

SERVICIOS DE GITHUB STUDENT DEVELOPER PACK ____ **7**

SEGURIDAD (ROLES RBAC, BCRYPT Y 2FA OTP) ____ **7**

CIFRADO EN TRÁNSITO Y EN REPOSO ____ **8**

HARDENING DE RED Y SEGURIDAD DEL SERVIDOR ____ **8**

CONSULTAS CRUD NATIVAS NOSQL Y AGREGACIONES ____ **8**

TIEMPO REAL, 36 ÍNDICES Y CLOUDINARY ____ **10**

DESPLIEGUE EN LA NUBE (DOCKER + RENDER + KEEP-ALIVE 24/7) ____ **11**

DIAGRAMAS DE DOCUMENTOS BSON Y SUBDOCUMENTOS ____ *12*

ESTRUCTURA DEL DOCUMENTO USUARIOS Y SUBDOCUMENTOS EMBEBIDOS (1:1) ____ **12**

ESTRUCTURA DEL DOCUMENTO PEDIDOS Y SU ARREGLO EMBEBIDO ITEMS[] (CERO JOINS) ____ **12**

ESTRUCTURA DEL DOCUMENTO CARRITOS Y SU ARREGLO EMBEBIDO ITEMS[] (0..N) ____ **13**

DOCUMENTOS DE CATÁLOGO COMERCIAL: PRODUCTOS Y CATEGORIAS ____ **13**

DOCUMENTOS DE INTERACCIÓN Y TRAZABILIDAD: RESENAS, NOTIFICACIONES E HISTORIAL_USUARIOS ____ **14**

DIAGRAMAS DE ARQUITECTURA DE MONGODB Y NUBE ____ *15*

DIAGRAMA DE ARQUITECTURA INTERNA DEL CLÚSTER MONGODB ATLAS (REPLICA SET DE 3 NODOS) ____ **15**

DIAGRAMA DE ARQUITECTURA CLOUD END-TO-END EN Producción (snackver.me) ____ **15**

DIAGRAMAS DE MODELO DE DATOS NOSQL Y RELACIONES (3 MÓDULOS SEPARADOS) ____ *16*

MÓDULO TRANSACCIONAL: FLUJO DE COMPRAS, CARRITO Y PEDIDOS (CERO JOINS) ____ **16**

MÓDULO DE CATÁLOGO Y VALORACIONES: CATEGORÍAS, PLATILLOS Y RESEÑAS ____ **17**

MÓDULO DE SEGURIDAD Y ALERTAS: AUDITORÍA FORENSE Y NOTIFICACIONES EN TIEMPO REAL 17

COMPARATIVA DE TRANSFORMACIÓN: MODELO RELACIONAL (MYSQL) VS MODELO DE DATOS NOSQL (MONGODB) 18

CATÁLOGO COMPLETO DE LOS 36 ÍNDICES EN MONGODB ATLAS 18

CATÁLOGO DE VISTAS PROGRAMADAS DEL SISTEMA *21*

PORTADA PRINCIPAL Y BUSCADOR GLOBAL DE PLATILLOS (INDEX.PHP) 21

CATÁLOGO DE PLATILLOS Y MENÚ POR CATEGORÍAS (FRONT-END/PRODUCTOS.PHP) 22

ACCESO DE USUARIOS Y MODAL DE VERIFICACIÓN 2FA OTP (FRONT-END/LOGIN.PHP) 22

REGISTRO DE CUENTAS Y VALIDACIÓN INSTITUCIONAL (FRONT-END/REGISTRO.PHP) 23

VITRINA DIGITAL Y MENÚ DE LA TIENDA ESTUDIANTIL (BACK-END/CLIENTE/BUSCARTIENDA.PHP Y BACK-END/CLIENTE/VER_TIENDA.PHP) 24

CARRITO DE COMPRAS Y CONTROL DE EXISTENCIAS (BACK-END/CLIENTE/CARRITO.PHP) 25

HISTORIAL DE PEDIDOS, COMPROBANTES SPEI (SIMULACION) Y CANCELACIONES (BACK-END/CLIENTE/MISPEDIDOS.PHP) 26

PANEL DE CONTROL DEL VENDEDOR CON KPIS EN VIVO (BACK-END/VENDEDOR/DASHBOARD_VENDEDOR.PHP) 27

INVENTARIO DE PLATILLOS Y CONTROL DE DISPONIBILIDAD (BACK-END/VENDEDOR/MISPLATILLOS.PHP) 27

DESPACHO Y GESTIÓN DE PEDIDOS EN TIEMPO REAL (BACK-END/VENDEDOR/PEDIDOS_PENDIENTES.PHP) 28

BITÁCORA DE AUDITORÍA FORENSE Y MONITOREO DE ACCESOS (BACK-END/ADMIN/HISTORIAL_USUARIOS.PHP) 28

MÓDULOS COMPLEMENTARIOS DEL SISTEMA WEB (RESUMEN DE LAS 24 VISTAS PROGRAMADAS) *29*

CONCLUSIONES *30*

INDICE DE IMÁGENES

Imagen 1 Colección usuarios: Estructura BSON y Subdocumentos Embebidos tienda, verificacion_otp y recuperacion_otp (Embedding 1:1). *12*

Imagen 2 Representativo BSON en la coleccion "Usuarios" *12*

Imagen 3 Colección pedidos: Estructura BSON y Arreglo Embebido items[] (Snapshot Histórico 1:N, Cero JOINs). _____ 12
Imagen 4 Representativo BSON en la colección "Pedidos" _____ 13
Imagen 5 Colección carritos: Estructura BSON y Arreglo Embebido items[] (Gestión Dinámica de Platillos). _ 13
Imagen 6 Colecciones de Catálogo: productos (Inventario y Stock Atómico) y categorías (Clasificación del Menú). _____ 13
Imagen 7 Colecciones de Interacción y Auditoría: resenas, notificaciones e historial_usuarios. _____ 14
Imagen 8 Arquitectura Interna de MongoDB Atlas: Replica Set de 3 Nodos, Motor WiredTiger y Cifrado AES-256. _____ 15
Imagen 9 Arquitectura Cloud End-to-End en Producción (Namecheap DNS/SSL + Render Docker + MongoDB Atlas + Cloudinary + Puente HTTPS 443). _____ 15
Imagen 10 Módulo Transaccional NoSQL: Flujo de Compras entre usuarios, carritos, pedidos y productos. __ 16
Imagen 11 Módulo de Catálogo y Reseñas: Clasificación, Publicación de Alimentos y Evaluación Comunitaria. _____ 17
Imagen 12 Módulo de Seguridad y Alertas: Auditoría Forense y Notificaciones Reactivas. _____ 17
Imagen 13 Portada Principal de Snackver (index.php): _____ 21
Imagen 14 Catálogo de Platillos (Productos.php) _____ 22
Imagen 15 Acceso al sistema (login.php) _____ 22
Imagen 16 Modal 2FA OTP _____ 23
Imagen 17 Verificación de código al Correo Electrónico _____ 23
Imagen 18 Entrando Después de la Verificación _____ 24
Imagen 19 Menú De La Tienda (BuscarTienda.php) _____ 24
Imagen 20 Vitrina del Vendedor (ver_tienda.php) _____ 25
Imagen 21 Carrito de Compras (carrito.php) _____ 25
Imagen 22 Historial de Pedidos (MisPedidos.php - Antes) _____ 26
Imagen 23 Historial de Pedidos (MisPedidos.php - Despues) _____ 26
Imagen 24 Dashboard del Vendedor (dashboard_vendedor.php) _____ 27
Imagen 25 Inventario y Catálogo del Vendedor (Misplatillos.php) _____ 27
Imagen 26 Despacho de Pedidos en Vivo (pedidos_pendientes.php _____ 28
Imagen 27 Bitácora de Auditoría Forense (historial_usuarios.php) _____ 28

INDICE DE TABLAS

Tabla 1 Consultas CRUD Y Agregación _____ 8
Tabla 2 Tabla Comparativa Modelo Relacional vs No Relacional _____ 18
Tabla 3 Catalogo de los 36 índices de Mongodb Atlas _____ 18
Tabla 4 Resumen de SnackVer _____ 29

INTRODUCCIÓN

Snackver ("Comida entre universitarios") es una plataforma web desarrollada para solucionar la coordinación de pedidos y entregas de alimentos dentro del Instituto Tecnológico de Veracruz. Su propósito central es conectar a los estudiantes que emprenden vendiendo alimentos con sus compañeros y docentes, agilizando la compra de comida en los recesos académicos.

El sistema operaba sobre una base de datos relacional MySQL local compuesta por 10 tablas normalizadas. Al llevar el proyecto a un entorno en la nube con usuarios concurrentes, este esquema presentó limitaciones evidentes: cuellos de botella y lentitud al requerir múltiples JOINs para armar el historial de compras, un esquema rígido que complicaba cambios rápidos en los platillos, almacenamiento local de fotos que se borraban al reiniciar contenedores Docker en la nube, y la falta de mecanismos modernos de doble factor de autenticación (2FA).

Se realizó una modernización integral sustituyendo por completo el motor relacional por MongoDB Atlas, organizando la información en 8 colecciones documentales NoSQL optimizadas con 36 índices B-Tree. Con esto se eliminó el 100% de las sentencias SQL en el código. Además, se aprovecharon beneficios del GitHub Student Developer Pack (clúster gestionado en la nube con MongoDB Atlas M0 y dominio propio snackver.me con certificado SSL emitido por Namecheap), se integró Cloudinary CDN para la gestión escalable de imágenes, se implementó 2FA OTP despachado por HTTPS hacia Google Apps Script, y se desplegó todo el entorno en un contenedor Docker en Render.

CONTENIDO: IMPLEMENTACION TECNICA

A continuación se describe qué se construyó en cada uno de los ocho requisitos solicitados por la rúbrica y cómo operan dentro del sistema:

MIGRACIÓN DE SQL A MONGODB ATLAS

Se transformó la arquitectura de datos de relacional a documental NoSQL para la base de datos de producción **sistemawebsnackver**:

- **Reestructuración Documental**

Las 10 tablas relacionales de MySQL se consolidaron en 8 colecciones BSON: usuarios, categorias, productos, pedidos, carritos, resenas, notificaciones e historial_usuarios. Se unificaron los tipos de usuario (cliente, vendedor y administrador) dentro de la colección usuarios diferenciados por el atributo "rol". La información del negocio del vendedor se integró como un subdocumento embebido (tienda), y las compras se almacenaron con sus platillos embebidos (items[]), evitando uniones de tablas.

- **Conector Oficial de MongoDB para PHP**

Se instaló la extensión nativa ext-mongodb v2.5 en PHP 8.2 y la librería oficial mongodb/mongodb ^2.5 mediante Composer, permitiendo manejar documentos BSON de forma nativa.

- **Patrón Singleton de Conexión**

En el archivo **Back-end/conexion/conexion_mongo.php** se implementó la clase ConexionMongo bajo el patrón de diseño Singleton. Esto asegura que exista una sola conexión activa hacia el clúster durante cada petición HTTP, reutilizando sockets y activando compresión de red Snappy y Zlib.

A continuación se muestra el comando de consola oficial de MongoDB Shell (mongosh) utilizado para conectarse al clúster en la nube con cifrado TLS 1.3, seleccionar la base de datos de producción y verificar el estado general de las colecciones migradas:

```
// 1. Conexión segura por consola mongosh hacia el clúster de producción en MongoDB Atlas:
mongosh "mongodb+srv://cluster0.ceqk0dl.mongodb.net/sistemawebsnackver?retryWrites=true&w=majority" --username admin_snackver

// 2. Selección de la base de datos y comprobación de colecciones migradas:
use sistemawebsnackver
show collections
db.stats()
```

SERVICIOS DE GITHUB STUDENT DEVELOPER PACK

Se activaron y configuraron dos servicios incluidos en el paquete de beneficios para estudiantes de GitHub:

- **Base de Datos en la Nube con MongoDB Atlas**

Se aprovisionó el clúster en la nube Cluster0 en la región us-east-1 (AWS), configurada como un Replica Set distribuido de 3 nodos (1 primario y 2 secundarios) con respaldo continuo y cifrado en disco.

- **Dominio Propio y Certificado SSL con Namecheap**

Se registró y vinculó el dominio oficial snackver.me, configurando los registros DNS correspondientes para apuntar al servidor de la aplicación y activando certificados SSL/TLS 1.3.

SEGURIDAD (ROLES RBAC, BCRYPT Y 2FA OTP)

Se programó un esquema de seguridad por capas para proteger las cuentas y la información de la comunidad:

- **Control de Acceso Basado en Roles**

Cada usuario posee un atributo rol ("cliente", "vendedor", "admin") en su documento de MongoDB. Al iniciar sesión, el sistema valida este rol en la sesión del servidor ($_SESSION["rol"]) antes de permitir el acceso a las vistas privadas.

- **Cifrado de Contraseñas con Bcrypt**

Las contraseñas de todos los usuarios se almacenan procesadas con password_hash(PASSWORD_BCRYPT) y se verifican con password_verify(). En ningún punto se guardan contraseñas en texto claro.

- **Autenticación en Dos Pasos (2FA OTP) en Dos Fases**

Para evitar que el usuario espere a que el servidor envíe un correo electrónico antes de ver la pantalla de verificación, el flujo se dividió en dos fases: en la Fase 1 (fase=generar), PHP genera un código numérico aleatorio de 6 dígitos con vigencia de 15 minutos, lo guarda en el subdocumento verificacion_otp en MongoDB y abre la ventana modal en la pantalla de inmediato. En la Fase 2 (fase=enviar_smtp), el navegador solicita en segundo plano el envío del correo.

- **Despacho de Correo por HTTPS en Puerto 443**

Dado que las plataformas en la nube como Render bloquean los puertos SMTP tradicionales (25, 465, 587) en cuentas gratuitas, se construyó EmailService.php para enviar los correos mediante peticiones HTTPS seguras por el puerto 443 hacia un Web App de Google Apps Script vinculado a Gmail.

Cada inicio de sesión, verificación OTP, cambio de clave o eliminación se registra de forma inmutable en la colección historial_usuarios con la dirección IP, el navegador y la fecha y hora exacta en formato UTC.

CIFRADO EN TRÁNSITO Y EN REPOSO

- **Cifrado en Tránsito**

Toda la comunicación web se realiza forzosamente por HTTPS bajo protocolo TLS 1.3 en https://snackver.me. La conexión entre el servidor web y MongoDB Atlas viaja cifrada mediante sockets TLS/SSL (tls=true), al igual que las comunicaciones con Cloudinary y Google Apps Script.

- **Cifrado en Reposo**

Los datos de las 8 colecciones documentales se encuentran protegidos en disco en los servidores de MongoDB Atlas bajo el motor WiredTiger mediante cifrado de grado bancario AES-256.

HARDENING DE RED Y SEGURIDAD DEL SERVIDOR

- **Control de Accesos al Clúster (IP Access List)**

El acceso a MongoDB Atlas se encuentra restringido a nivel de red y protegido por autenticación segura de usuarios con el mecanismo SCRAM-SHA-256.

- **Blindaje con Apache (.htaccess)**

Se configuraron reglas de servidor que bloquean el acceso directo a archivos sensibles como .env, .git y directorios internos con error 403 Forbidden, deshabilitan la visualización de directorios (Options -Indexes) y activan compresión Gzip (mod_deflate).

- **Aislamiento de Variables de Configuración**

Las contraseñas de base de datos y llaves de API nunca están escritas dentro del código fuente; en desarrollo se leen desde un archivo .env protegido mediante vlucas/phpdotenv y en producción se configuran como variables de entorno seguras en Render.

CONSULTAS CRUD NATIVAS NOSQL Y AGREGACIONES

A continuación se presenta la correspondencia de las operaciones nativas de MongoDB utilizadas en los módulos del sistema:

Tabla 1 Consultas CRUD Y Agregación

OPERACIÓN NoSQL	MÉTODOS Y OPERADORES DE MONGODB	VISTAS Y MÓDULOS DONDE SE EJECUTA
CREATE (Inserción)	insertOne() con ObjectId y UTCDateTime	registro.php (usuarios), nuevoplatillo.php (productos), checkout.php (pedidos con items[]), categorias.php, resenas.php, HistorialService y NotificacionService.
READ (Consulta)	find(), findOne(), countDocuments(), Regex, sort	index.php, Productos.php, resultados_busqueda.php, BuscarTienda.php, ver_tienda.php, MisPedidos.php, pedidos_pendientes.php, clientes.php, historial_usuarios.php.

OPERACIÓN NoSQL	MÉTODOS Y OPERADORES DE MONGODB	VISTAS Y MÓDULOS DONDE SE EJECUTA
UPDATE (Actualización)	updateOne(), updateMany() con $set, $inc, $push, $pull	procesar_pedido.php ($inc en stock), cancelar_pedido_cliente.php ($inc restauración), carrito.php ($push/$pull items[]), editar_platillo.php, MiTienda.php, Miperfil.php.
DELETE (Eliminación)	deleteOne(), deleteMany() en cascada	eliminar_cliente.php (usuario + carrito), eliminar_vendedor.php (vendedor + platillos), Misplatillos.php (baja de platillo).
AGGREGATE (Analític)	aggregate() con $match, $group, $sum, $avg, $facet	dashboard_admin.php (métricas globales con $facet), dashboard_vendedor.php (3 KPIs en tiempo real), index.php y ver_tienda.php (promedios de calificación).

A continuación se presentan los comandos documentales de mongosh implementados en los controladores del Back-end para las operaciones críticas del ciclo comercial, evidenciando la manipulación atómica de arreglos embebidos ($push, $pull) y el control de inventario ($inc):

```
// 1. CREATE: Inserción de pedido congelando los platillos en items[] (checkout.php - Cero JOINs)
db.pedidos.insertOne({
  id_pedido_num: 1045,
  id_cliente: ObjectId("6718a910e4b0a1c2d3e4f502"),
  id_vendedor: ObjectId("6718a910e4b0a1c2d3e4f501"),
  items: [
    { id_producto: ObjectId("6718aa00e4b0a1c2d3e4f600"), nombreproducto: "Torta de Cochinita", cantidad: 2,
precio_unitario: 45.0, importe: 90.0 }
  ],
  total: 90.0,
  estadopedido: "En preparación",
  metodopago: "Transferencia SPEI",
  fechapedido: new Date()
})
```

```
// 2. UPDATE: Descuento atómico de existencias al confirmar la compra (procesar_pedido.php)
db.productos.updateOne(
  { _id: ObjectId("6718aa00e4b0a1c2d3e4f600"), stock_disponible: { $gte: 2 } },
  { $inc: { stock_disponible: -2 } }
)

// 3. UPDATE: Agregado atómico de platillo al carrito único del estudiante (carrito.php)
db.carritos.updateOne(
  { id_cliente: ObjectId("6718a910e4b0a1c2d3e4f502") },
  { $push: { items: { id_producto: ObjectId("6718aa00e4b0a1c2d3e4f600"), cantidad: 1, agregado_en: new Date() } } }
)
```

Asimismo, la plataforma implementa pipelines de agregación nativa (aggregate) para calcular analítica en tiempo real y promedios de calificación comunitaria sin sobrecargar el servidor web:

```
// 4. AGGREGATE: Cálculo de promedio de calificación y total de opiniones (index.php / ver_tienda.php)
db.resenas.aggregate([
  { $match: { id_producto: ObjectId("6718aa00e4b0a1c2d3e4f600") } },
  { $group: {
      _id: "$id_producto",
      calificacion_promedio: { $avg: "$calificacion" },
      total_opiniones: { $sum: 1 }
    }
  }
])

// 5. AGGREGATE: Cálculo de KPIs del Vendedor en vivo (dashboard_vendedor.php)
db.pedidos.aggregate([
  { $match: { id_vendedor: ObjectId("6718a910e4b0a1c2d3e4f501"), estadopedido: { $ne: "Cancelado" } } },
  { $group: {
      _id: "$id_vendedor",
      pedidos_atendidos: { $sum: 1 },
      ingresos_totales: { $sum: "$total" }
    }
  }
])
```

TIEMPO REAL, 36 ÍNDICES Y CLOUDINARY

- **Sincronización**

Se programó un motor reactivo en sincronizar_tiempo_real.php y tiempo_real.js que monitorea el estado de la vista cada 2 segundos. El servidor calcula un hash MD5 de los datos; si no hay cambios (status="unchanged"), no se transfiere contenido ni se toca el DOM. Si hay cambios, la vista se refresca de inmediato en 0 ms sin recargar la página.

- **Campana de Alertas y Métricas Reactivas**

El script notificaciones.js consulta cada 1.5 segundos para mantener actualizados el contador de la campana, la lista de pedidos nuevos y los 3 indicadores numéricos del vendedor.

- **Gestión y Optimización de Imágenes con Cloudinary**

Al seleccionar una foto de comida, el navegador muestra una vista previa instantánea con URL.createObjectURL y la comprime a través de un elemento HTML5 <canvas> a ~100 KB antes de enviarla. La imagen se guarda en Cloudinary CDN y se entrega optimizada en formatos WebP/AVIF con f_auto,q_auto.

A continuación se detallan los comandos de mongosh empleados para la creación de los índices B-Tree en MongoDB Atlas, así como el comando de diagnóstico explain() que demuestra la sustitución de escaneos completos (COLLSCAN) por búsquedas indexadas (IXSCAN):

```
// 1. Creación de índices B-Tree de rendimiento en consola mongosh:
db.usuarios.createIndex({ email: 1 }, { unique: true, name: "uniq_email" })
db.carritos.createIndex({ id_cliente: 1 }, { unique: true, name: "uniq_carrito_cliente" })
db.pedidos.createIndex({ id_vendedor: 1, estadopedido: 1, fechapedido: -1 }, { name: "idx_pedidos_ven_est_fecha" })
db.categorias.createIndex({ estado: 1, nombrecategoria: 1 }, { name: "idx_cat_estado_nombre" })

// 2. Comprobación del plan de ejecución para verificar el uso del índice:
db.pedidos.find({
  id_vendedor: ObjectId("6718a910e4b0a1c2d3e4f501"),
  estadopedido: "En preparación"
}).sort({ fechapedido: -1 }).explain("executionStats")
```

DESPLIEGUE EN LA NUBE (DOCKER + RENDER + KEEP-ALIVE 24/7)

- **Contenedor Docker para Producción**

Se construyó un Dockerfile sobre la imagen oficial php:8.2-apache instalando las extensiones PECL requeridas (mongodb, gd, zip, curl) y configurando los módulos de Apache.

- **Ajustes PHP de Producción (snackver.ini)**

Se configuró output_buffering = 4096 y display_errors = Off en /usr/local/etc/php/conf.d/snackver.ini para asegurar que los controladores de tiempo real siempre entreguen respuestas JSON limpias sin advertencias.

- **Repositorio Privado y Despliegue Continuo (CI/CD)**

El repositorio de código fuente se mantiene en modalidad Privada en GitHub para proteger el desarrollo. Render se conecta directamente mediante su aplicación oficial Render GitHub App y despliega automáticamente cualquier cambio en la rama main.

- **Disponibilidad Continua 24/7**

Se configuró un monitor programado en cron-job.org que envía una petición HTTP al sitio cada 10 minutos, evitando que la instancia gratuita de Render se suspenda por inactividad.

DIAGRAMAS DE DOCUMENTOS BSON Y SUBDOCUMENTOS

A continuación se presentan los diagramas de estructura BSON para cada colección en formato UML técnico puro, con tipografía clara y relaciones directas:

ESTRUCTURA DEL DOCUMENTO USUARIOS Y SUBDOCUMENTOS EMBEBIDOS (1:1)

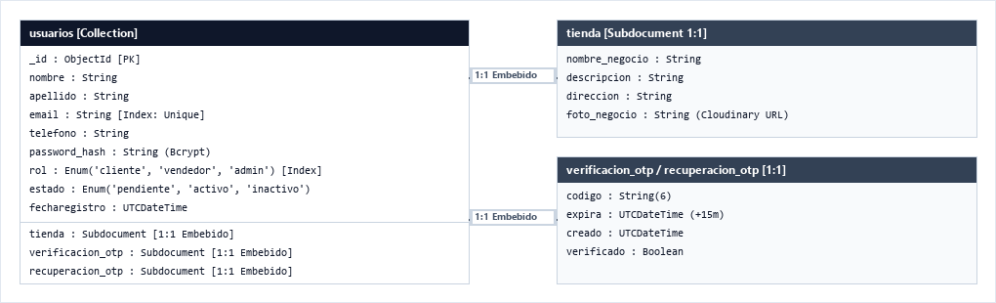


Imagen 1 Colección usuarios: Estructura BSON y Subdocumentos Embebidos tienda, verificacion_otp y recuperacion_otp (Embedding 1:1).

La colección usuarios reúne a clientes, vendedores y administradores en una sola estructura. En lugar de tablas separadas, la información de la tienda del alumno y los códigos de seguridad para 2FA se guardan como subdocumentos embebidos dentro de la misma cuenta, permitiendo leer todo el perfil en una sola operación:



Imagen 2 Representativo BSON en la coleccion "Usuarios"

ESTRUCTURA DEL DOCUMENTO PEDIDOS Y SU ARREGLO EMBEBIDO ITEMS[] (CERO JOINS)



Imagen 3 Colección pedidos: Estructura BSON y Arreglo Embebido items[] (Snapshot Histórico 1:N, Cero JOINs).

En la colección pedidos se eliminó la necesidad de tablas secundarias de detalle. Cada pedido contiene un arreglo embebido llamado items[] donde se guardan el nombre, la cantidad y el precio exacto al momento de la compra. Esto garantiza que si el vendedor cambia de precio el platillo en el futuro, los pedidos anteriores no se alteran y el historial se consulta en una sola lectura sin JOINs:

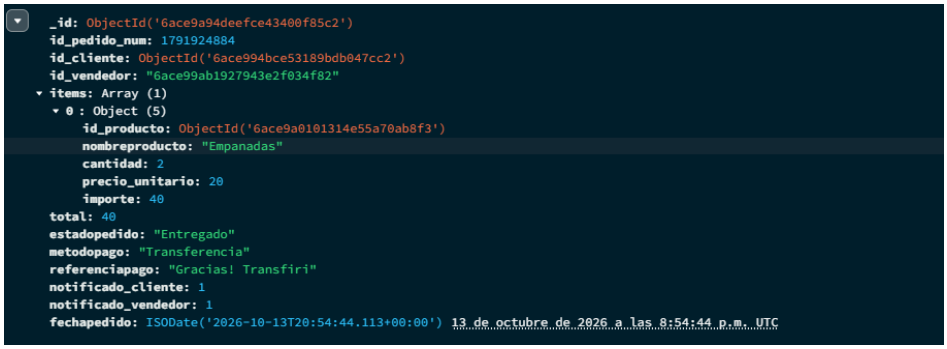


Imagen 4 Representativo BSON en la colección "Pedidos"

ESTRUCTURA DEL DOCUMENTO CARRITOS Y SU ARREGLO EMBEBIDO ITEMS[] (0..N)

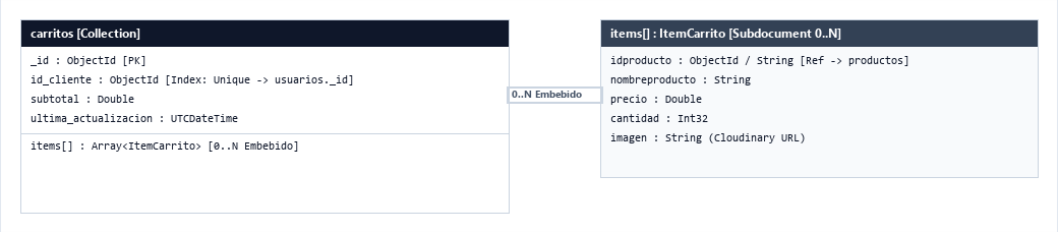


Imagen 5 Colección carritos: Estructura BSON y Arreglo Embebido items[] (Gestión Dinámica de Platillos).

Cada estudiante cuenta con un único carrito activo garantizado por el índice único uniq_carrito_cliente ({ id_cliente: 1 }). Los platillos seleccionados se agregan o quitan atómicamente dentro del arreglo items[] usando los operadores de MongoDB $push, $pull y $set.

DOCUMENTOS DE CATÁLOGO COMERCIAL: PRODUCTOS Y CATEGORIAS

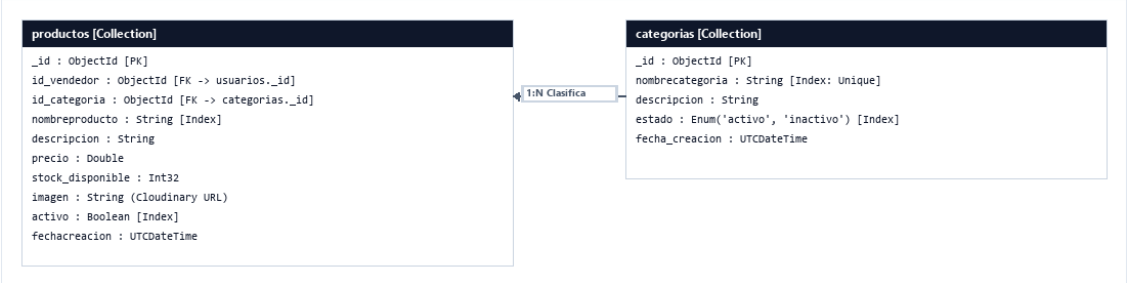


Imagen 6 Colecciones de Catálogo: productos (Inventario y Stock Atómico) y categorías (Clasificación del Menú).

La colección productos maneja los platillos publicados por cada vendedor, sus existencias en tiempo real y la foto en Cloudinary. El inventario se descuenta de forma segura con el operador atómico $inc. La colección categorias organiza los tipos de comida con un índice compuesto para mostrar las categorías activas de inmediato.

DOCUMENTOS DE INTERACCIÓN Y TRAZABILIDAD: RESENAS, NOTIFICACIONES E HISTORIAL_USUARIOS

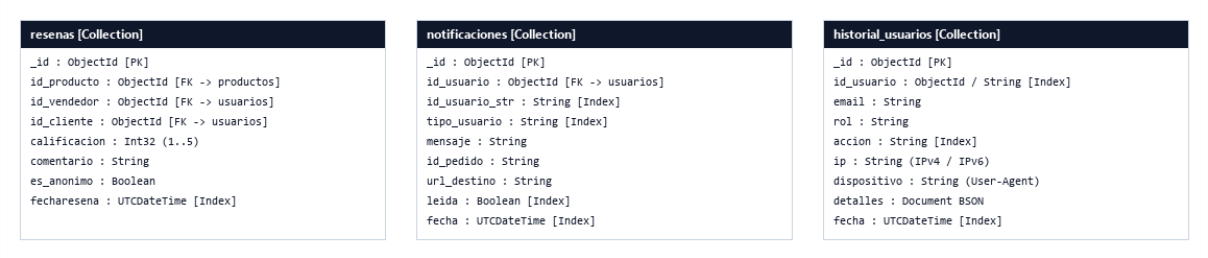


Imagen 7 Colecciones de Interacción y Auditoría: resenas, notificaciones e historial_usuarios.

La colección resenas permite que los alumnos califiquen a los vendedores con estrellas y comentarios (públicos o anónimos). Notificaciones alimenta la campana de alertas en tiempo real, e historial_usuarios registra los eventos de seguridad para auditoría.

DIAGRAMAS DE ARQUITECTURA DE MONGODB Y NUBE

A continuación, se ilustra la arquitectura de la base de datos distribuida y el flujo de los servicios en la nube:

DIAGRAMA DE ARQUITECTURA INTERNA DEL CLÚSTER MONGODB ATLAS (REPLICA SET DE 3 NODOS)

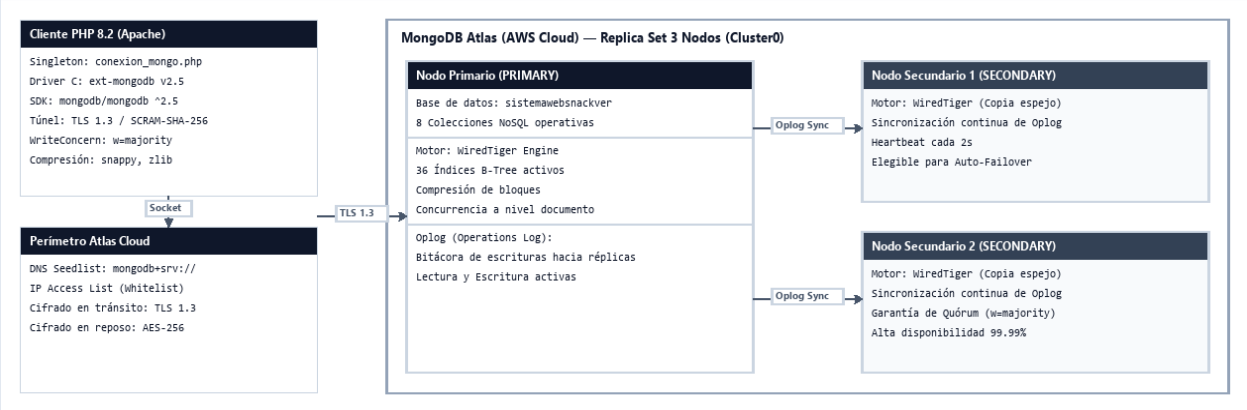


Imagen 8 Arquitectura Interna de MongoDB Atlas: Replica Set de 3 Nodos, Motor WiredTiger y Cifrado AES-256.

- **Capa de Aplicación:** PHP 8.2 bajo el conector Singleton ConexionMongo, con compresión Snappy/Zlib y confirmación de escritura mayoritaria (w=majority).
- **Perímetro de Seguridad:** Cadena DNS segura mongodb+srv://, lista de acceso por IP, credenciales SCRAM-SHA-256 y canal cifrado TLS 1.3.
- **Replica Set de 3 Nodos:** Un nodo Primario que procesa todas las lecturas y escrituras, replicando los cambios mediante el registro Oplog a dos nodos Secundarios con verificación periódica (heartbeat cada 2s) y recuperación automática ante fallas.
- **Motor de Almacenamiento WiredTiger:** Administra los 36 índices B-Tree en memoria RAM y aplica compresión de disco y cifrado AES-256 en reposo.

DIAGRAMA DE ARQUITECTURA CLOUD END-TO-END EN Producción (snackver.me)

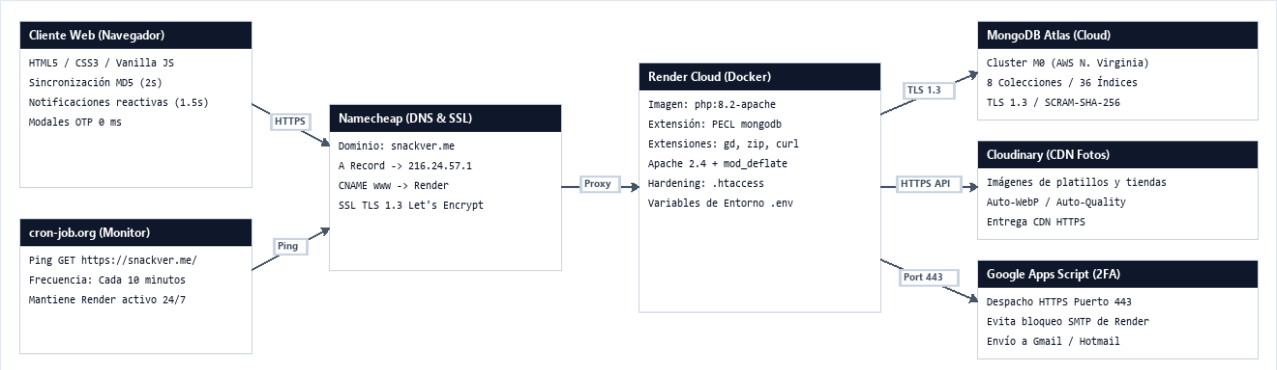


Imagen 9 Arquitectura Cloud End-to-End en Producción (Namecheap DNS/SSL + Render Docker + MongoDB Atlas + Cloudinary + Puente HTTPS 443).

- **Acceso Seguro del Usuario:** Los estudiantes ingresan mediante el dominio oficial https://snackver.me administrado en Namecheap con certificado SSL TLS 1.3.
- **Servidor de Aplicación en la Nube:** Contenedor Docker sobre php:8.2-apache en Render Web Services con compresión mod_deflate y reglas de seguridad en .htaccess.
- **Servicios Integrados en la Nube:** MongoDB Atlas (datos NoSQL), Cloudinary CDN (entrega de imágenes en formatos WebP/AVIF) y Google Apps Script (envío de correos 2FA por HTTPS en puerto 443).
- **Disponibilidad 24/7:** Monitor en cron-job.org que envía una petición cada 10 minutos para mantener el servidor web activo continuamente.

DIAGRAMAS DE MODELO DE DATOS NOSQL Y RELACIONES (3 MÓDULOS SEPARADOS)

El modelo de datos NoSQL se divide en tres flujos funcionales para facilitar su comprensión, complementados por la tabla comparativa de migración y el catálogo de 36 índices:

MÓDULO TRANSACCIONAL: FLUJO DE COMPRAS, CARRITO Y PEDIDOS (CERO JOINS)

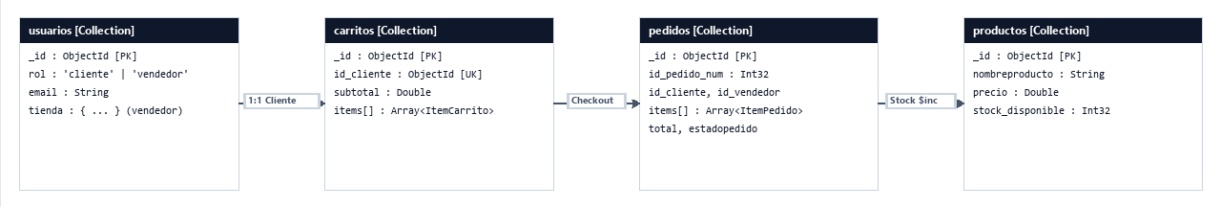


Imagen 10 Módulo Transaccional NoSQL: Flujo de Compras entre usuarios, carritos, pedidos y productos.

- **Relación 1:1 Cliente a Carrito:** Garantizada por el índice único uniq_carrito_cliente ({ id_cliente: 1 }), evitando carritos duplicados.
- **Checkout Inmutable:** Al confirmar la compra, los productos del carrito se copian directamente dentro del arreglo items[] en pedidos, congelando los precios sin depender de consultas posteriores a productos.
- **Descuento Atómico de Existencias ($inc):** El stock de los alimentos se descuenta de inmediato con el operador $inc: { stock_disponible: -cantidad }. Si el alumno cancela el pedido, el sistema devuelve las existencias automáticamente con el valor positivo.

MÓDULO DE CATÁLOGO Y VALORACIONES: CATEGORÍAS, PLATILLOS Y RESEÑAS



Imagen 11 Módulo de Catálogo y Reseñas: Clasificación, Publicación de Alimentos y Evaluación Comunitaria.

- **Clasificación:** Cada producto hace referencia a su id_categoria, permitiendo clasificar los platillos en el menú.
- **Publicación del Vendedor:** Los platillos se vinculan mediante id_vendedor con el usuario correspondiente y guardan la URL segura de Cloudinary.
- **Opiniones y Calificaciones:** La colección resenas vincula al cliente, al vendedor y al platillo, permitiendo calcular el promedio de estrellas mediante pipelines de agregación ($avg).

MÓDULO DE SEGURIDAD Y ALERTAS: AUDITORÍA FORENSE Y NOTIFICACIONES EN TIEMPO REAL

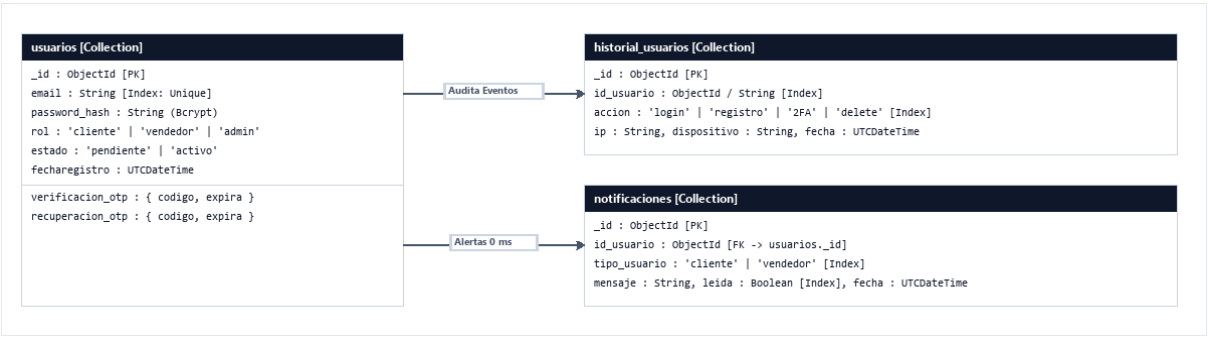


Imagen 12 Módulo de Seguridad y Alertas: Auditoría Forense y Notificaciones Reactivas.

- **Auditoría Forense:** Cada acción importante (inicio de sesión, cambio de clave, validación 2FA o eliminación) se guarda en historial_usuarios con la dirección IP y el navegador.
- **Alertas al Instante:** Cuando cambia el estado de un pedido, se inserta una alerta en notificaciones y el script notificaciones.js actualiza la campana del usuario en 1.5 segundos.

COMPARATIVA DE TRANSFORMACIÓN: MODELO RELACIONAL (MYSQL) VS MODELO DE DATOS NOSQL (MONGODB)

Tabla 2 Tabla Comparativa Modelo Relacional vs No Relacional

MODELO ANTERIOR (10 TABLAS MYSQL)	MODELO ACTUAL (8 COLECCIONES MONGODB)	TRANSFORMACIÓN Y BENEFICIO NOSQL OBTENIDO
cliente + vendedor + Admin	1. usuarios	Unificación de cuentas con discriminador "rol" y subdocumentos embebidos tienda y OTP.
categoriaproducto	2. categorias	Clasificación ágil referenciada por id_categoria con índice compuesto por estado y nombre.
producto	3. productos	Catálogo con imágenes en Cloudinary CDN y control de existencias atómico con $inc.
pedido + detallepedido	4. pedidos	Arreglo embebido items[] dentro de cada pedido. Cero JOINs y precios congelados.
carrito + detallecarrito	5. carritos	Arreglo embebido items[] en un único documento por alumno (uniq_carrito_cliente).
resena	6. resenas	Calificaciones con soporte para comentarios públicos o anónimos (es_anonimo).
notificaciones	7. notificaciones	Alertas de compras indexadas por destinatario y estado de lectura.
(No existía en MySQL)	8. historial_usuarios	Bitácora de seguridad forense con IP, navegador y fecha UTC.

CATÁLOGO COMPLETO DE LOS 36 ÍNDICES EN MONGODB ATLAS

Para garantizar respuestas inmediatas y evitar escaneos completos de colecciones (COLLSCAN), se configuraron los siguientes 36 índices B-Tree en MongoDB Atlas:

Tabla 3 Catalogo de los 36 índices de Mongodb Atlas

COLECCIÓN	NOMBRE DEL ÍNDICE	CLAVES BSON	TIPO	CONSULTA O VISTA QUE ACELERA
usuarios	uniq_email	{ email: 1 }	Único	Login instantáneo y correos no duplicados.
usuarios	idx_rol	{ rol: 1 }	Simple	Filtrado por rol (cliente/vendedor/admin).
usuarios	idx_usuarios_rol_estado	{ rol: 1, estado: 1 }	Compuesto	Paneles de administración y tiendas activas.

COLECCIÓN	NOMBRE DEL ÍNDICE	CLAVES BSON	TIPO	CONSULTA O VISTA QUE ACELERA
categorias	idx_nombrecategoria	{ nombrecategoria: 1 }	Simple	Búsqueda por nombre de categoría.
categorias	idx_cat_estado_nombre	{ estado: 1, nombrecategoria: 1 }	Compuesto	Categorías activas ordenadas en index.php.
productos	idx_vendedor	{ id_vendedor: 1 }	Simple	Catálogo propio del vendedor en Misplatillos.php.
productos	idx_categoria	{ id_categoria: 1 }	Simple	Filtrado por categoría en Productos.php.
productos	idx_activo	{ activo: 1 }	Simple	Exclusión de platillos inactivos.
productos	idx_prod_vendedor_stock	{ id_vendedor: 1, stock_disponible: 1 }	Compuesto	Inventario y existencias por vendedor.
productos	idx_prod_cat_stock	{ id_categoria: 1, stock_disponible: 1 }	Compuesto	Platillos disponibles por categoría.
productos	idx_prod_estado	{ estado: 1 }	Simple	Estado en consultas de administración.
productos	idx_prod_estado_vendedor	{ estado: 1, id_vendedor: 1 }	Compuesto	Platillos por vendedor y estado.
pedidos	idx_pedidos_cliente	{ id_cliente: 1 }	Simple	Historial de compras del estudiante.
pedidos	idx_pedidos_vendedor	{ id_vendedor: 1 }	Simple	Bandeja de pedidos del vendedor.
pedidos	idx_pedidos_fecha	{ fechapedido: -1 }	Simple	Orden cronológico global de órdenes.
pedidos	idx_pedidos_cli_notif	{ id_cliente: 1, notificado_cliente: 1 }	Compuesto	Sondeo reactivo de estado para cliente.
pedidos	idx_pedidos_ven_notif	{ id_vendedor: 1, notificado_vendedor: 1 }	Compuesto	Detección reactiva de nuevos pedidos.
pedidos	idx_pedidos_cli_fecha	{ id_cliente: 1, fechapedido: -1 }	Compuesto	Historial del cliente en MisPedidos.php.
pedidos	idx_pedidos_ven_estado	{ id_vendedor: 1, estadopedido: 1 }	Compuesto	KPI de pedidos pendientes del vendedor.
pedidos	idx_pedidos_estado	{ estado: 1 }	Simple	Agregaciones en dashboard_admin.php.

COLECCIÓN	NOMBRE DEL ÍNDICE	CLAVES BSON	TIPO	CONSULTA O VISTA QUE ACELERA
pedidos	idx_pedidos_ven_fecha	{ id_vendedor: 1, fechapedido: -1 }	Compuesto	Órdenes recientes en dashboard_vendedor.php.
pedidos	idx_pedidos_ven_est_fecha	{ id_vendedor: 1, estadopedido: 1, fechapedido: -1 }	Compuesto	Filtro en 0 ms en pedidos_pendientes.php.
pedidos	idx_pedidos_cli_est_fecha	{ id_cliente: 1, estadopedido: 1, fechapedido: -1 }	Compuesto	Filtro en 0 ms en MisPedidos.php.
carritos	uniq_carrito_cliente	{ id_cliente: 1 }	Único	Un único carrito activo por estudiante.
carritos	idx_carritos_cliente	{ id_cliente: 1 }	Simple	Lectura y actualización en carrito.php.
resenas	idx_resena_producto	{ id_producto: 1 }	Simple	Promedios y opiniones por platillo.
resenas	idx_resena_cliente	{ id_cliente: 1 }	Simple	Reseñas emitidas en resenas.php.
resenas	idx_resenas_fecha	{ fecharesena: -1 }	Simple	Carrusel de reseñas en index.php.
historial_usuarios	idx_historial_fecha	{ fecha: -1 }	Simple	Auditoría cronológica descendente.
historial_usuarios	idx_historial_usuario_fecha	{ id_usuario: 1, fecha: -1 }	Compuesto	Auditoría por usuario específico.
historial_usuarios	idx_historial_accion_fecha	{ accion: 1, fecha: -1 }	Compuesto	Filtro por acción (login, 2FA, delete).
notificaciones	id_usuario_1	{ id_usuario: 1 }	Simple	Recuperación de alertas por usuario.
notificaciones	fecha_-1	{ fecha: -1 }	Simple	Orden cronológico descendente.
notificaciones	idx_notif_usr_tipo_fecha	{ id_usuario: 1, tipo_usuario: 1, fecha: -1 }	Compuesto	Campana de alertas sin ordenamiento en RAM.
notificaciones	idx_notif_usrstr_tipo_fecha	{ id_usuario_str: 1, tipo_usuario: 1, fecha: -1 }	Compuesto	Búsquedas por ID de usuario en texto.
notificaciones	idx_notif_usr_leida	{ id_usuario: 1, tipo_usuario: 1, leida: 1 }	Compuesto	Conteo inmediato de badge rojo no leído.

CATÁLOGO DE VISTAS PROGRAMADAS DEL SISTEMA

A continuación, se documentan las vistas reales del sistema Snackver (https://snackver.me). En cada apartado se explica qué se realizó en la pantalla, qué información observa el usuario, cómo interactúa y qué operaciones en MongoDB Atlas se ejecutan en segundo plano, acompañado de su respectiva captura de pantalla:

PORTADA PRINCIPAL Y BUSCADOR GLOBAL DE PLATILLOS (INDEX.PHP)

Se programó la página de inicio institucional de Snackver como el centro de descubrimiento de alimentos para la comunidad del campus. La interfaz cuenta con una cabecera hero con llamada a la acción, un buscador global en vivo que permite buscar alimentos por nombre o ingrediente, un carrusel dinámico de categorías gastronómicas y una sección interactiva de reseñas recientes dejadas por estudiantes.

En esta vista, el estudiante puede escribir cualquier término en el buscador para ver sugerencias inmediatas, hacer clic en una categoría para filtrar el menú, o explorar la vitrina de platillos destacados. Detrás de escena, la página consulta las colecciones categorias (con el índice idx_cat_estado_nombre) y productos, y ejecuta un pipeline de agregación sobre resenas junto con usuarios para promediar las estrellas y mostrar el nombre del comensal.



Imagen 13 Portada Principal de Snackver (index.php):

CATÁLOGO DE PLATILLOS Y MENÚ POR CATEGORÍAS (FRONT-END/PRODUCTOS.PHP)

Se construyó una cuadrícula comercial totalmente responsiva donde los estudiantes pueden explorar la oferta gastronómica disponible en el campus. Cada tarjeta de platillo muestra la fotografía del alimento optimizada desde Cloudinary CDN, el nombre comercial, el precio en pesos mexicanos, el número de existencias disponibles y la tienda del vendedor que lo prepara.

El alumno puede seleccionar la cantidad deseada y hacer clic en el botón de agregar al carrito, visualizando una confirmación visual instantánea. A nivel de base de datos, esta pantalla ejecuta una consulta con filtro indexado sobre la colección productos usando el índice compuesto idx_prod_cat_stock ({ id_categoria: 1, stock_disponible: 1 }), descartando platillos agotados o inactivos y vinculando la información comercial del vendedor.

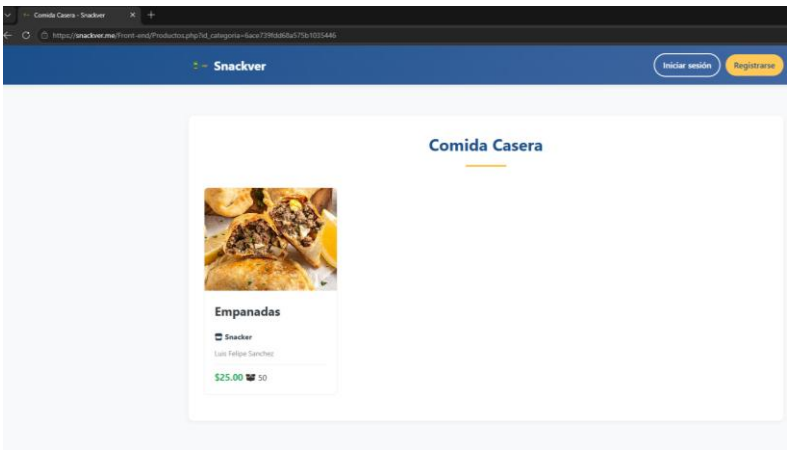


Imagen 14 Catálogo de Platillos (Productos.php)

ACCESO DE USUARIOS Y MODAL DE VERIFICACIÓN 2FA OTP (FRONT-END/LOGIN.PHP)

Se desarrolló el módulo de inicio de sesión con soporte de seguridad de doble factor de autenticación (2FA). La pantalla solicita el correo electrónico institucional y la contraseña del usuario. Si la cuenta requiere verificación por código OTP, la interfaz despliega en la ventana modal flotante sobre el formulario, solicitando los 6 dígitos numéricos sin recargar la página.

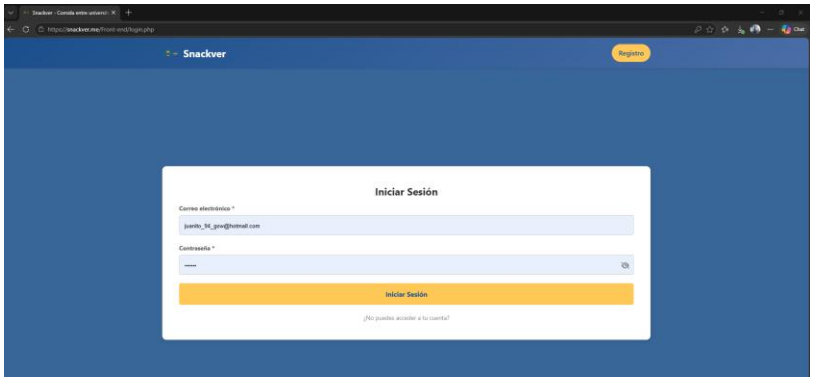


Imagen 15 Acceso al sistema (login.php)

REGISTRO DE CUENTAS Y VALIDACIÓN INSTITUCIONAL (FRONT-END/REGISTRO.PHP)

Se implementó el formulario de registro donde los alumnos pueden darse de alta como Clientes o como Vendedores universitarios. En caso de elegir el perfil de Vendedor, la pantalla despliega campos adicionales para capturar el nombre de su negocio, descripción y punto de entrega dentro de la institución.

Al enviar el formulario, el sistema comprueba que el correo no exista en usuarios, cifra la contraseña con Bcrypt y genera el documento del usuario insertando el subdocumento embebido tienda (si es vendedor) y el subdocumento verificacion_otp con el código de activación. La modal de verificación de 6 dígitos se abre y el usuario recibe el correo de bienvenida.

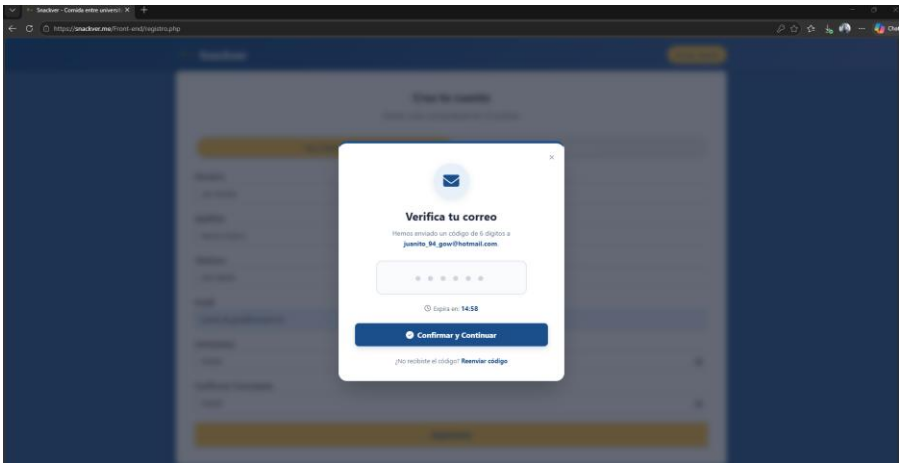


Imagen 16 Modal 2FA OTP

Verificación del código en el Correo ingresado al Registrar:

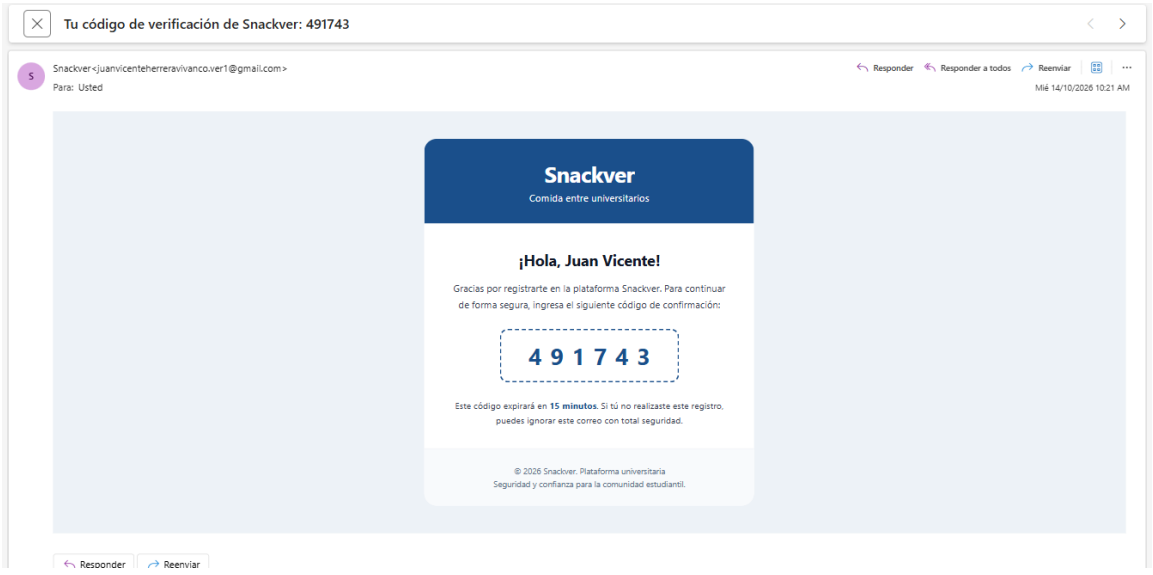


Imagen 17 Verificación de código al Correo Electrónico

Cuando el usuario presiona "Iniciar Sesión", el servidor busca el correo en la colección usuarios mediante el índice único uniq_email y verifica la contraseña utilizando password_verify() contra el hash Bcrypt. Si es correcto, el servidor genera un código OTP de 6 dígitos que guarda en el subdocumento verificacion_otp y responde al navegador para abrir la modal de inmediato, al mismo tiempo, despacha en segundo plano el correo electrónico mediante HTTPS hacia Google Apps Script. El evento de inicio de sesión queda registrado en historial_usuarios:

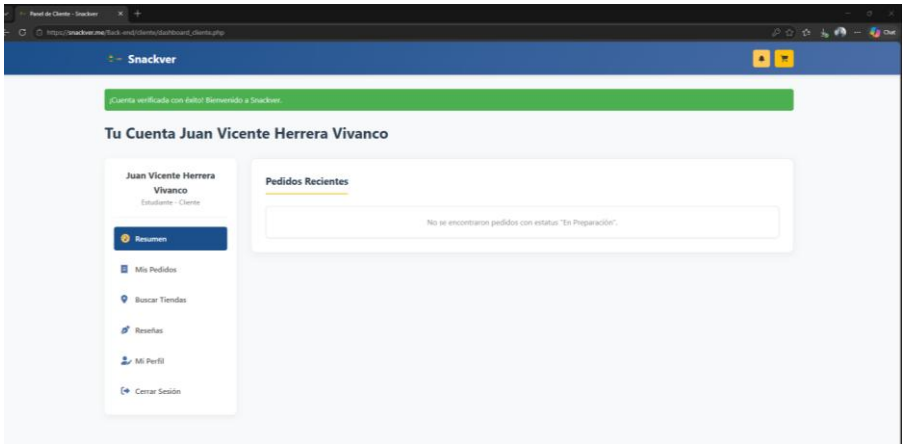


Imagen 18 Entrando Después de la Verificación

VITRINA DIGITAL Y MENÚ DE LA TIENDA ESTUDIANTIL (BACK-END/CLIENTE/BUSCARTIENDA.PHP Y BACK-END/CLIENTE/VER_TIENDA.PHP)

Se diseñó la vista de escaparate digital para cada puesto o negocio de comida administrado por estudiantes. La interfaz presenta la portada y foto del negocio, el nombre comercial, la ubicación física de entrega el promedio de satisfacción obtenido en estrellas y la lista completa de platillos que ofrece.

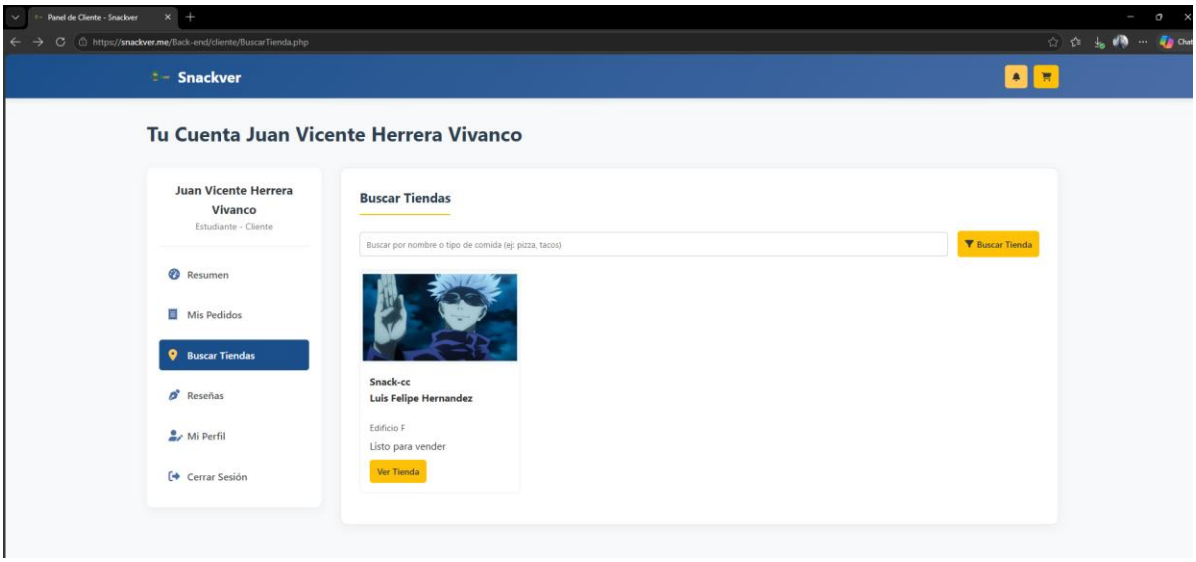


Imagen 19 Menú De La Tienda (BuscarTienda.php)

El comensal puede consultar el menú específico de este vendedor y añadir productos al carrito de forma directa. En MongoDB Atlas, esta vista lee el subdocumento tienda dentro del usuario vendedor, consulta la colección productos filtrando por el índice idx_vendedor, y ejecuta una agregación con el operador $avg sobre resenas para calcular la calificación promediO.

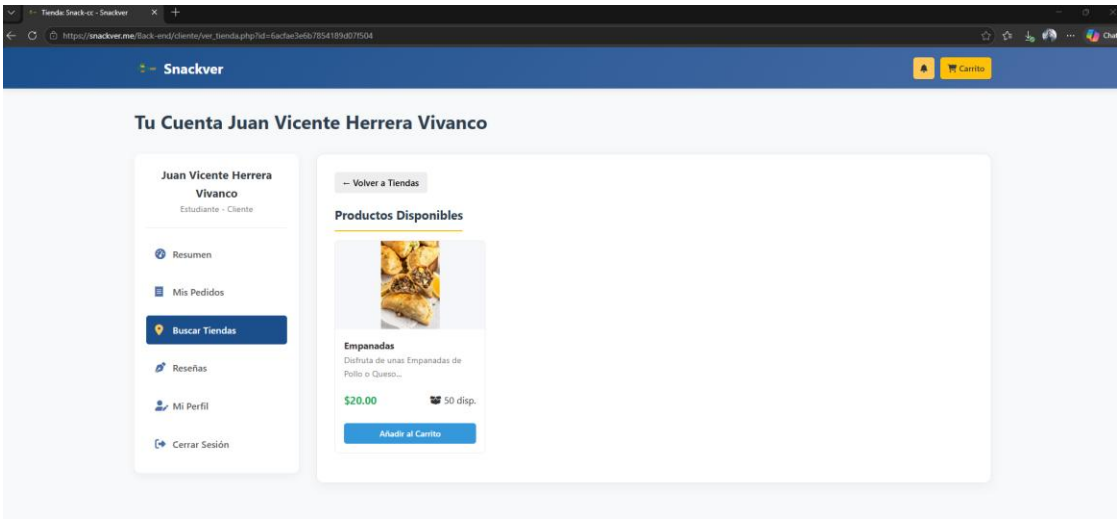


Imagen 20 Vitrina del Vendedor (ver_tienda.php)

CARRITO DE COMPRAS Y CONTROL DE EXISTENCIAS (BACK-END/CLIENTE/CARRITO.PHP)

Se programó el módulo de gestión del carrito de compras para los alumnos. La vista muestra la lista de alimentos seleccionados, la foto miniatura, el precio unitario, controles para incrementar (+) o decrementar (-) la cantidad, el importe por renglón y el cálculo dinámico del total a pagar, además de validar que no se exceda el inventario disponible.

El alumno puede ajustar las cantidades o eliminar platillos con un clic. En la base de datos, cada alumno posee un único documento en la colección carritos indexado por uniq_carrito_cliente. Las modificaciones se aplican directamente en el arreglo embebido items[] mediante operaciones $push, $pull y $set, garantizando manipulación atómica de los productos sin bloqueos.

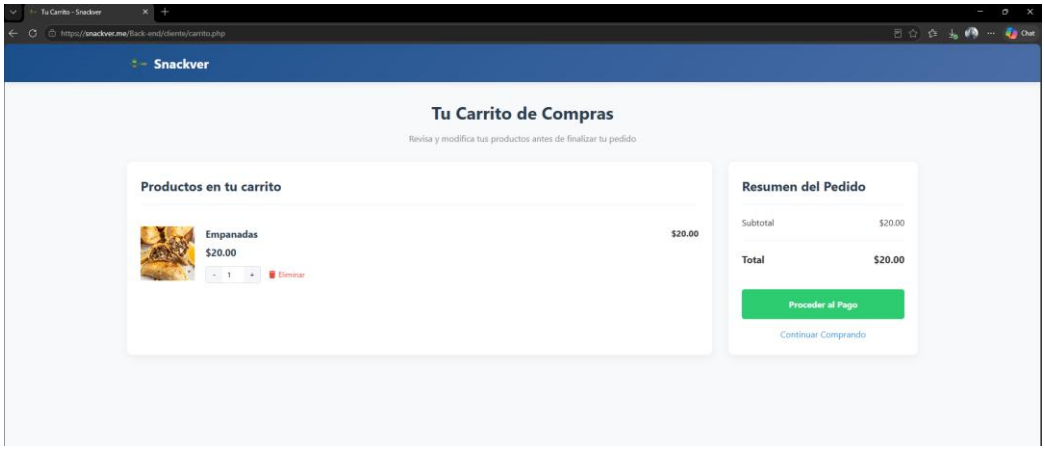


Imagen 21 Carrito de Compras (carrito.php)

HISTORIAL DE PEDIDOS, COMPROBANTES SPEI (SIMULACION) Y CANCELACIONES (BACK-END/CLIENTE/MISPEDIDOS.PHP)

Se construyó el centro de seguimiento de pedidos del estudiante comensal. La interfaz organiza las compras con indicadores visuales por semáforo de color según su estado ("En preparación", "Listo para entrega", "Entregado" o "Cancelado"). Permite registrar el número de autorización o comprobante en caso de pagos por transferencia SPEI, y cuenta con un botón para cancelar pedidos pendientes.

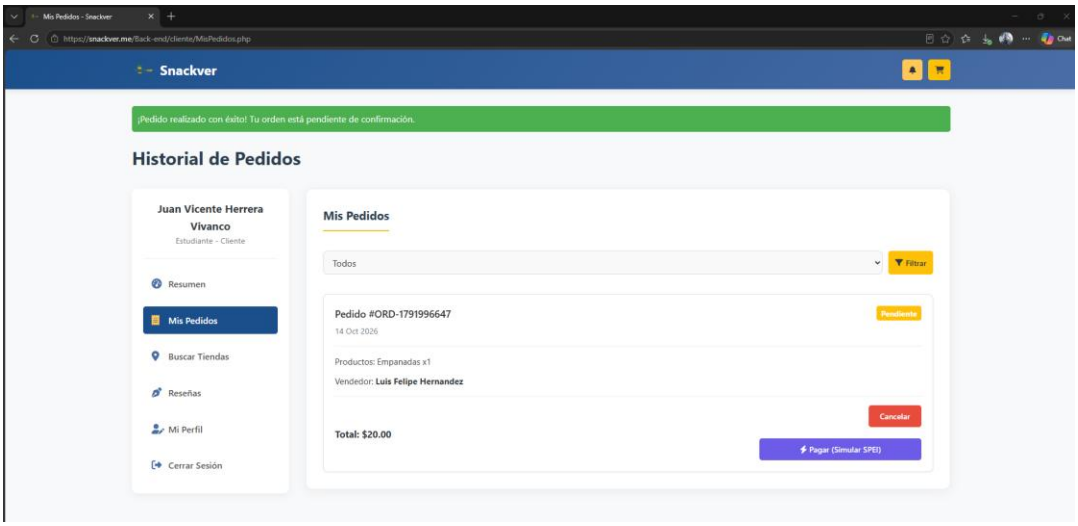


Imagen 22 Historial de Pedidos (MisPedidos.php - Antes)

La consulta se resuelve en una sola lectura sin uniones (Cero JOINs) gracias a que cada documento de la colección pedidos contiene el arreglo items[] con todos los platillos y precios congelados. Si el alumno cancela un pedido en preparación, el sistema actualiza el estado a "Cancelado" y ejecuta una operación atómica $inc en la colección productos para devolver de inmediato las existencias al inventario del vendedor.

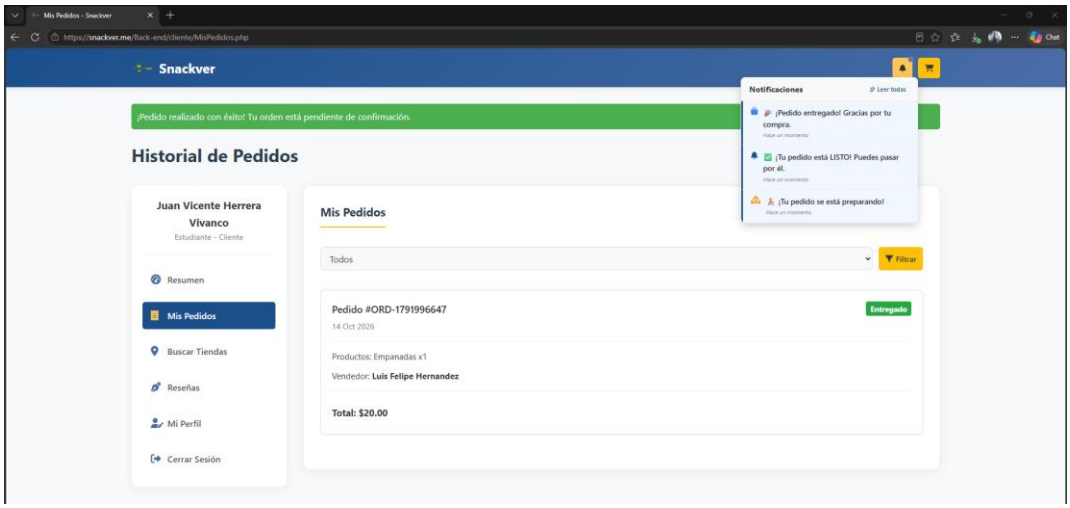


Imagen 23 Historial de Pedidos (MisPedidos.php - Despues)

PANEL DE CONTROL DEL VENDEDOR CON KPIS EN VIVO (BACK-END/VENDEDOR/DASHBOARD_VENDEDOR.PHP)

Se implementó el panel de mando para el estudiante emprendedor. También incluye una campana de notificaciones con contador rojo y una lista de los pedidos recibidos recientemente.

El script notificaciones.js consulta cada 1.5 segundos en segundo plano. Si ingresa una nueva orden, el contador de la campana se actualiza y en vivo sin necesidad de que el vendedor recargue la página. En Atlas, esto se logra mediante consultas indexadas con idx_pedidos_ven_estado y agregaciones sobre pedidos.

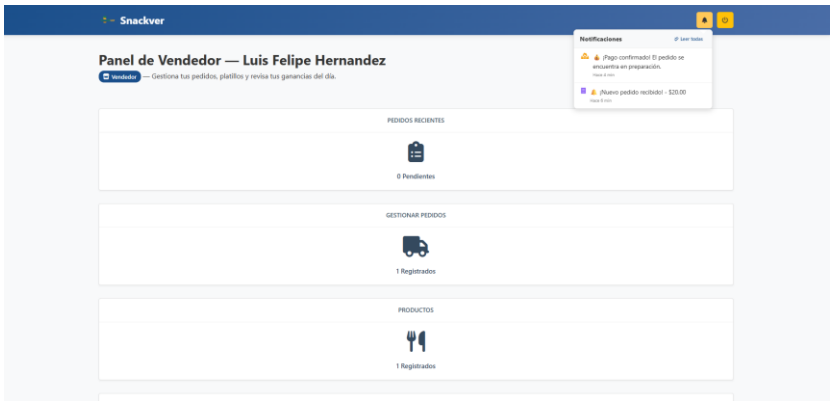


Imagen 24 Dashboard del Vendedor (dashboard_vendedor.php)

INVENTARIO DE PLATILLOS Y CONTROL DE DISPONIBILIDAD (BACK-END/VENDEDOR/MISPLATILLOS.PHP)

Se programó la pantalla de administración de catálogo e inventario para los vendedores. La vista lista todos los alimentos publicados por el usuario con su imagen de Cloudinary, categoría, precio y stock disponible. Cuenta con botones directos para agregar un nuevo platillo, editar la información.

Cuando el vendedor añade o modifica un platillo, el formulario procesa la imagen en el cliente mediante un <canvas> de HTML5 para comprimirla a ~100 KB antes de subirla a Cloudinary vía CloudinaryService.php. La base de datos guarda la URL segura HTTPS y gestiona el inventario en la colección productos con el índice idx_prod_vendedor_stock.

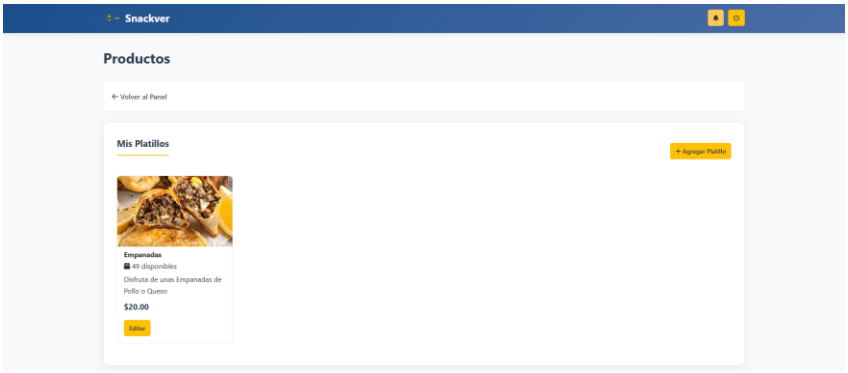


Imagen 25 Inventario y Catálogo del Vendedor (Misplatillos.php)

DESPACHO Y GESTIÓN DE PEDIDOS EN TIEMPO REAL (BACK-END/VENDEDOR/PEDIDOS_PENDIENTES.PHP)

Se desarrolló la bandeja de despacho de comandas para que el vendedor gestione la preparación de comida durante los recesos. La pantalla muestra los pedidos clasificados por pestañas ("Pendientes", "En preparación", "Listos", "Entregados"), con botones rápidos de acción para avanzar el pedido al siguiente estado con un solo clic.

Al cambiar el estado de un pedido (por ejemplo, a "Listo para entrega"), el servidor actualiza el campo estadopedido en la colección pedidos y genera automáticamente un documento en notificaciones. El cliente que espera su comida recibe la alerta sonora y visual en su dispositivo. El filtro entre pestañas se ejecuta de forma instantánea mediante el índice compuesto idx_pedidos_ven_est_fecha.

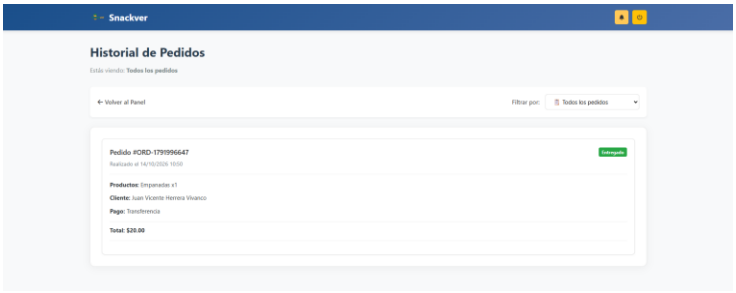


Imagen 26 Despacho de Pedidos en Vivo (pedidos_pendientes.php

BITÁCORA DE AUDITORÍA FORENSE Y MONITOREO DE ACCESOS (BACK-END/ADMIN/HISTORIAL_USUARIOS.PHP)

Se implementó el panel de seguridad y auditoría para el administrador del sistema. La interfaz presenta una tabla cronológica con el registro detallado de cada evento de seguridad ocurrido en la plataforma: inicios de sesión, registros nuevos, solicitudes de código 2FA OTP, cambios de clave y bajas de usuarios.

Cada fila muestra el nombre del usuario, su correo electrónico, el rol, la acción ejecutada, la dirección IP desde donde se conectó, el navegador y sistema operativo (User-Agent) y la fecha y hora exacta en formato UTC. En MongoDB Atlas, esta vista consulta la colección historial_usuarios mediante los índices idx_historial_fecha e idx_historial_accion_fecha, permitiendo una rápida auditoría ante cualquier incidente de seguridad.

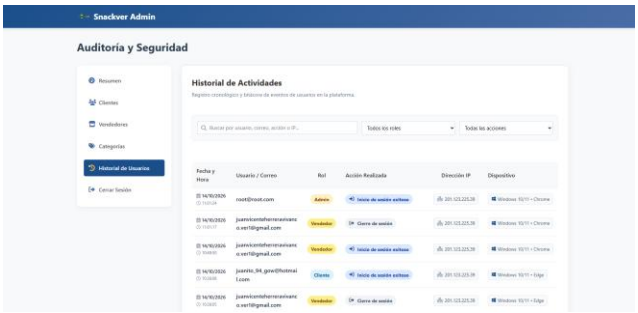


Imagen 27 Bitácora de Auditoría Forense (historial_usuarios.php)

MÓDULOS COMPLEMENTARIOS DEL SISTEMA WEB (RESUMEN DE LAS 24 VISTAS PROGRAMADAS)

Además de las 11 vistas principales documentadas con captura de pantalla, Snackver cuenta con 13 interfaces adicionales completamente programadas y conectadas a MongoDB Atlas en producción, conformando el ecosistema de 24 vistas del proyecto:

Tabla 4 Resumen de SnackVer

MÓDULO DEL SISTEMA	VISTAS COMPLEMENTARIAS PROGRAMADAS	OPERACIÓN Y PROPÓSITO TÉCNICO
Módulo Público y Ayuda	Front-end/footer/contacto.php, preguntas-clientes.php, preguntas-vendedores.php, guia-cliente.php, guia-vendedor.php, terminos.php	6 páginas de soporte, políticas universitarias y guías paso a paso para el uso correcto de la plataforma.
Módulo de Seguridad	recuperacion_password.php, cambiar_password.php, verificar_codigo.php	Flujo de recuperación de contraseñas mediante código OTP en modal flotante (0 ms) y validación en Atlas.
Módulo Cliente	BuscarTienda.php, checkout.php, Miperfil.php, resenas.php	Directorio de tiendas con buscador, confirmación de pedido con descuento atómico, perfil y formulario de reseñas.
Módulo Vendedor	nuevoplatillo.php, editar_platillo.php, MiTienda.php	Formularios con compresión <canvas> para alta y edición de platillos, y configuración del puesto del alumno.
Módulo Administrador	dashboard_admin.php, clientes.php, vendedores.php, categorias.php	Métricas analíticas globales con $facet, gestión y baja en cascada de usuarios, y catálogo de categorías.

CONCLUSIONES

La modernización de SnackVer realizada en la Unidad 2 permitió mejorar la forma en que funciona la plataforma, pasando de una base de datos MySQL a MongoDB Atlas. Con este cambio se obtuvieron varios beneficios:

- **Organización de la información:** Se utilizaron ocho colecciones en MongoDB para guardar los datos de forma más flexible. Algunos elementos, como los productos de los pedidos, se almacenan juntos, lo que facilita la consulta de la información y reduce la necesidad de relacionar varias tablas.
- **Mejor experiencia para el usuario:** Se agregaron funciones para actualizar la información sin recargar toda la página, mostrar notificaciones y comprimir las fotografías antes de subirlas. También se implementó un sistema de verificación mediante códigos OTP para reforzar el inicio de sesión.
- **Mayor seguridad:** Se incorporaron diferentes medidas de protección, como roles y permisos para los usuarios, contraseñas cifradas con Bcrypt, verificación en dos pasos y conexiones seguras mediante HTTPS. Además, se agregó un historial para registrar los cambios realizados en las cuentas.
- **Uso de servicios en la nube:** Se integraron herramientas como MongoDB Atlas para almacenar la información, Render para alojar la aplicación y un servicio de monitoreo para revisar su disponibilidad. También se configuró el dominio snackver.me con certificado SSL, facilitando el acceso seguro a la plataforma.